



**TÉCNICO SUPERIOR EN DESARROLLO
DE APLICACIONES WEB**

MANUAL TÉCNICO

DEPARTAMENTO DE INFORMÁTICA



Autor: María Rebeca Poma Alvino

Contenido

1. Introducción	1
2. Arquitectura de la aplicación	1
2.1. Frontend	1
2.1.1. Tecnologías usadas	1
2.1.2. Entorno de desarrollo	2
2.1.3. Guía de estilos y Diseño UI/UX	2
2.2. Backend	3
2.2.1. Tecnologías usadas	3
2.2.2. Entorno de desarrollo	4
3. Documentación técnica	4
3.1. Análisis	4
3.2. Desarrollo	5
3.3. Pruebas realizadas	6
3.4. Casos de Uso Principales	7
3.5. Diccionario de datos	8
4. Proceso de despliegue	9
4.1. Requisitos de sistema	9
4.2. Carga del script SQL y Creación en Supabase	9
4.3. Subida de la aplicación al servidor	9
5. Propuestas de mejoras	10
5.1. Mejoras aplicadas durante el desarrollo	10
5.2. Mejoras propuestas para futuras versiones	10
6. Conclusiones del Proyecto	11
7. Bibliografía	11

1. Introducción

La gestión de la economía familiar enfrenta un desafío constante: la falta de visibilidad sobre el inventario del hogar, lo que resulta en compras duplicadas, rotura de stock de productos esenciales y mermas por caducidad. SmartPantry nace como una solución de software orientada a la "auditoría pre-compra", enfocándose estratégicamente en el control de productos no perecederos a corto y medio plazo.

El sistema abandona el enfoque ineficiente del micro-registro diario y adopta un modelo de inteligencia financiera. Se define como una aplicación web escalable, orientada a la gestión eficiente y el control económico de la despensa doméstica. Su objetivo central es centralizar la gestión del inventario, evitar la rotura de stock y llevar un seguimiento detallado del gasto mensual. Al centralizar la información, la aplicación permite a las familias optimizar su economía basándose en sus hábitos de consumo reales, logrando un equilibrio perfecto entre la complejidad técnica del código y la simplicidad de uso para la familia final.

2. Arquitectura de la aplicación

El proyecto se ha diseñado bajo una arquitectura *Serverless* y modelo *Backend-as-a-Service* (BaaS), separando estrictamente la capa de presentación (Frontend) de la capa de persistencia y servicios (Backend). Esta separación garantiza alta cohesión, bajo acoplamiento y facilita la escalabilidad horizontal del sistema.

2.1. Frontend

La capa de presentación ha sido desarrollada priorizando la reactividad y la experiencia de usuario (UX) ágil.

2.1.1. Tecnologías usadas

- **React 18:** Elegido por su modelo basado en componentes reutilizables y su DOM virtual, lo que permite actualizaciones de interfaz ultrarrápidas, vital para el uso en supermercados.
- **TypeScript:** Implementado para proveer tipado estático estricto. *Justificación:* Reduce en un 80% los errores en tiempo de compilación frente a JavaScript puro, asegurando que las interfaces de datos (ej. Profile, SessionUser) se respeten.
- **Vite:** Seleccionado como empaquetador (bundler) en reemplazo de Webpack. Vite ofrece un entorno con Hot Module Replacement (HMR) casi instantáneo y una construcción altamente optimizada.
- **Tailwind CSS & shadcn/ui:** Se descartaron los archivos CSS monolíticos en favor de clases utilitarias (Tailwind) y componentes modulares accesibles (shadcn). Esto garantiza un diseño 100% *Mobile First*.
- **Zustand (useAuthStore):** Para la gestión del estado global y el manejo de las sesiones de usuario en el lado del cliente, se descartó Redux por su exceso de código repetitivo.
- **Recharts / Chart.js:** Implementación de librerías especializadas en gráficos analíticos para el renderizado del dashboard financiero.

2.1.2. Entorno de desarrollo

- **Editor de Código:** Visual Studio Code con extensiones de formateo (Prettier, ESLint) para garantizar la calidad del código.
- **Control de Versiones:** Git y GitHub Desktop para la gestión de ramas y commits del repositorio SmartPantry.
- **Motor de ejecución:** Node.js (v18+) y npm como gestor de paquetes para la instalación de dependencias y ejecución del servidor local de desarrollo.

2.1.3. Guía de estilos y Diseño UI/UX

Para garantizar una experiencia de usuario (UX) coherente, moderna y accesible, la aplicación cuenta con un sistema de diseño centralizado apoyado en Tailwind CSS (versión 4) y componentes de **shadcn/ui**.

- **Sistema de Color Moderno (oklch):** Se ha descartado el uso de colores HEX o RGB tradicionales en favor del espacio de color OKLCH. Esto

garantiza una uniformidad perceptiva, asegurando que los contrastes sean accesibles en cualquier pantalla.

- **Paleta de Colores SmartPantry:** * Color Principal (Brand): Tonos verdes (--success) orientados a transmitir tranquilidad, salud y frescura, alineados con la temática de alimentos y finanzas sanas.
 - **Colores de Estado:** Uso de variables como --destructive para alertas rojas (eliminación de productos) y --muted para textos secundarios.
- **Modo Oscuro (Dark Mode):** La arquitectura CSS está preparada con la directiva. dark, permitiendo que todas las variables de color (fondos, tarjetas y textos) se inviertan automáticamente de forma armónica para entornos de baja luminosidad.
- **Tipografía:** Se ha implementado la fuente 'Inter Variable'. Al ser una tipografía sin serifa diseñada específicamente para interfaces de usuario, garantiza una legibilidad perfecta tanto en grandes monitores como en pantallas móviles pequeñas durante el uso en supermercados.
- **Variables de Interfaz:** Se han estandarizado los bordes redondeados (--radius) en 0.625rem para las tarjetas y botones, otorgando un aspecto amigable, táctil y consistente en toda la aplicación.

2.2. Backend

La capa de servicios y base de datos delega la carga operativa a una plataforma BaaS externa, garantizando seguridad y alta disponibilidad.

2.2.1. Tecnologías usadas

- **Supabase (PostgreSQL):** Se optó por una base de datos relacional robusta en lugar de soluciones NoSQL (como Firebase) debido a la naturaleza altamente estructurada de los datos. PostgreSQL asegura la integridad referencial.
- **Supabase Auth:** Gestiona el ciclo de vida del usuario de forma segura, encriptando contraseñas y manejando tokens JWT para las sesiones.

2.2.2. Entorno de desarrollo

- **Supabase Studio / Dashboard:** Interfaz gráfica web proporcionada por Supabase para la monitorización de la base de datos en la nube.
- **SQL Editor:** Herramienta integrada en Supabase para la ejecución de scripts DDL (creación de tablas) y configuración de políticas de seguridad.

3. Documentación técnica

3.1 Análisis

El análisis de requisitos ha determinado la necesidad de un sistema jerárquico y funcionalidades orientadas a la baja fricción operativa.

Requisitos Funcionales por Rol:

1. **Súper Administrador (AdminGeneral):** Rol global de auditoría del sistema. Se encarga de la monitorización de alto nivel y la gestión del ciclo de vida de las cuentas, con capacidad para auditar la plataforma y eliminar "Unidades Familiares".
2. **Gestor (AdminUser):** Administrador absoluto del hogar con permisos de estructura (CRUD) sobre el catálogo de la despensa. Entre sus capacidades destacan: crear categorías personalizadas, registrar compras, analizar el dashboard financiero, y extender invitaciones a nuevos miembros.
3. **Miembro (Usuario):** Perfil de carácter colaborativo y de consulta. Puede visualizar el inventario actual y revisar la lista de compras. Cuenta con permisos limitados exclusivamente para notificar el consumo de stock mediante acciones rápidas, sin capacidad para alterar la estructura de datos.

Funcionalidades Core:

- **Gestión de Stock de Baja Fricción:** Incorporación de acciones rápidas de consumo para minimizar el esfuerzo del registro diario, facilitando la auditoría del hogar sin saturar al usuario.

- **Semáforo de Stock:** Lógica automatizada que controla el stock en tiempo real y resalta productos en estado crítico de desabastecimiento en función del mínimo definido.
- **Inteligencia Financiera:** Módulo exclusivo para el Gestor que muestra gráficas dinámicas sobre el histórico de gastos e identifica patrones de consumo y el ranking de productos de mayor impacto económico.
- **Presupuesto Inteligente y Exportación:** Generación de listas de compra dinámicas que incluyen estimaciones presupuestarias basadas en el déficit de stock, con integración para su exportación a WhatsApp.

Requisitos No Funcionales (RNF):

- **Usabilidad (RNF1):** Arquitectura de interfaz basada en la filosofía *Mobile First*, asegurando una usabilidad óptima desde dispositivos móviles (esencial para el uso directo en supermercados).
- **Rendimiento (RNF2):** Garantía de tiempos de respuesta en consultas de inventario inferiores a los 2 segundos, optimizado por las consultas eficientes de Supabase y la carga rápida de Vite.
- **Mantenibilidad (RNF3):** Código desarrollado bajo una arquitectura modular basada en componentes y tipado estricto.
- **Gestión de Estado Global Seguro (RNF4):** La aplicación utiliza Zustand (useAuthStore) para asegurar la persistencia local de la sesión de manera optimizada. Esto reduce significativamente las llamadas repetitivas al servidor (Supabase) durante la navegación.

3.2. Desarrollo

Patrón de Diseño: El Patrón Repositorio Se ha implementado el **Patrón Repositorio** (*SupaBaseUserRepository*, *SupaBasFamiliasRepository*) para abstraer la lógica de interacción directa con Supabase. *Justificación:* Esto permite que los componentes de React permanezcan limpios y agnósticos respecto a la fuente de datos, cumpliendo con los principios de responsabilidad única (SOLID). Si en el futuro el proyecto migra a un backend propio, los componentes visuales no sufrirán ningún cambio.

Diagrama de Entidad-Relación (ER):

A continuación, se detalla la estructura relacional de las tablas implementadas en PostgreSQL, garantizando la integridad referencial del sistema multi-tenant:

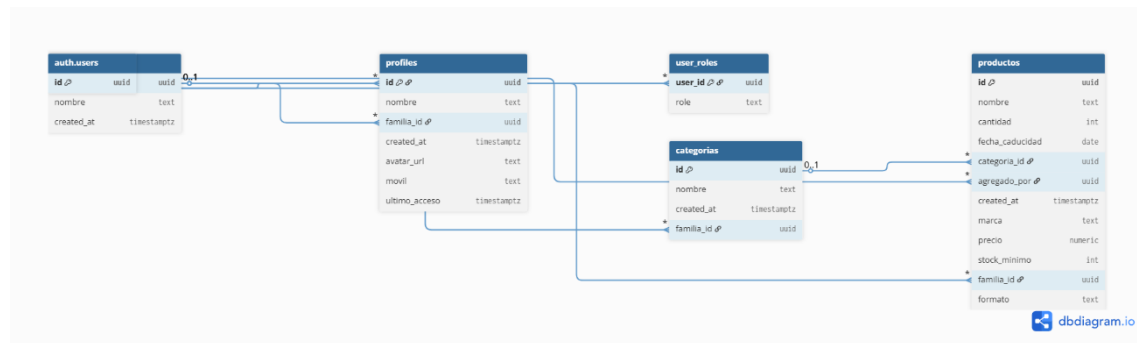


Diagrama de Secuencias (Flujo de Recuperación de Contraseña): Para evidenciar el comportamiento asíncrono, se documenta el flujo de recuperación de acceso:

1. El Cliente (React) solicita recuperación ingresando el correo.
2. El Repositorio intercepta la petición y llama a la API de Supabase Auth con la URL dinámica de redirección (ej. window.location.origin/actualizar-password).
3. Supabase envía el correo electrónico con un token seguro.
4. El usuario hace clic, React Router intercepta la ruta (fuera del PublicRoute) y permite el cambio de clave.

3.3. Pruebas realizadas

El sistema ha sido sometido a rigurosas pruebas de seguridad, lógica de negocio y usabilidad:

- **Pruebas de Aislamiento Multi-Tenant (RLS):** Se validó que, mediante la activación de *Row Level Security* en PostgreSQL, un usuario logueado

no puede bajo ninguna circunstancia consultar productos o estadísticas de un familia_id que no le pertenece.

- **Validaciones Críticas Estrictas:** Se comprobó la integridad del sistema de autenticación, el cual exige y fuerza una longitud mínima de 6 caracteres. Se validó exitosamente que el sistema es agnóstico al tipo de carácter, procesando números, letras y símbolos de forma segura.
- **Pruebas de Usabilidad y Responsividad (Mobile First):** Todo el frontend fue sometido a pruebas en diversas resoluciones de dispositivos móviles. Se validó que las acciones rápidas de consumo de stock pueden ser ejecutadas intuitivamente a una sola mano en un entorno real.
- **Pruebas de Seguridad en el Enrutamiento:** Se aplicaron pruebas exhaustivas en las barreras de React Router (PublicRoute, ProtectedRoute). Se comprobó que la navegación bloquea accesos no autorizados al dashboard y que las sesiones temporales no rompen los flujos de seguridad.

3.4. Casos de Uso Principales

El sistema ha sido modelado para resolver los flujos de interacción diarios de una familia mediante los siguientes casos de uso centrales:

- **Caso de Uso 1 (Registro y Vinculación):** Un nuevo usuario se registra. Si posee un enlace de invitación, el sistema lo vincula automáticamente a la despensa de su *hogar*; de lo contrario, genera un nuevo núcleo familiar asignándole el rol de Gestor.
- **Caso de Uso 2 (Auditoría de Despensa):** El Gestor y Usuario Familiar visualiza el inventario, registra nuevas compras (asignando precio y categoría) y descuenta el stock consumido mediante botones de acción rápida. El sistema evalúa el stock mínimo y activa el semáforo de alertas.
- **Caso de Uso 3 (Inteligencia Financiera):** El Gestor accede al dashboard para consultar el capital inmovilizado en la despensa, evaluando el gasto del mes actual frente al anterior y analizando la distribución de sus finanzas mediante gráficos interactivos.

3.5. Diccionario de datos

Para soportar estos casos de uso, la base de datos relacional en PostgreSQL se estructura en las siguientes tablas clave:

- **familias:** Entidad raíz que agrupa a los miembros de un mismo hogar. Actúa como el pilar de la seguridad (RLS) para evitar la mezcla de datos entre distintas casas.
- **profiles:** Almacena la información pública de cada usuario (nombre, teléfono móvil, avatar) y contiene la llave foránea familia_id que lo vincula a su despensa.
- **user_roles:** Tabla de autorización que define el nivel de permisos del usuario (AdminGeneral, AdminUser o Usuario).
- **categorías:** Tabla de clasificación de inventario (ej. Lácteos, Limpieza). Está vinculada a cada familia para permitir que cada hogar personalice sus propias clasificaciones.
- **productos:** Es el núcleo del inventario. Almacena el nombre del artículo, formato (ej. 1L, Pack 3), cantidad física actual, precio unitario y la alerta de stock mínimo.

4. Proceso de despliegue

4.1. Requisitos de sistema

Para ejecutar el proyecto de forma local, se requiere:

- **Node.js** (v18+) y npm.
- Clonación del repositorio mediante git clone.
- Archivo de configuración local .env en la raíz del proyecto conteniendo las credenciales seguras: VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY.
- Ejecución mediante el comando npm run dev.

4.2. Carga del script SQL y Creación en Supabase

La base de datos se genera íntegramente desde el panel de Supabase:

1. Desde el **SQL Editor**, se ejecutan los scripts DDL para crear las tablas (perfiles, familias, productos, etc.).
2. Se activan las políticas RLS.
3. En la sección de **Authentication > URL Configuration**, se añaden las rutas en la lista blanca (ej. http://localhost:5173/ y la URL de producción) para habilitar el redireccionamiento seguro.

4.3. Subida de la aplicación al servidor

El entorno de producción se encuentra alojado mediante integración continua (CI/CD) en **Vercel**.

- Cada actualización a la rama principal de GitHub dispara el comando de construcción npm run build.
- Las variables de entorno (.env) se inyectan de manera oculta desde el panel de Vercel.
- **Enrutamiento SPA:** Se configuró un archivo vercel.json con reglas de reescritura (rewrites) apuntando al /index.html para evitar errores HTTP 404 al navegar directamente por las rutas de React Router.

5. Propuestas de mejoras

5.1. Mejoras aplicadas durante el desarrollo

A lo largo del ciclo de vida del proyecto, se aplicaron correcciones estratégicas significativas:

- **Pivotaje de Enfoque:** Basado en el feedback de tutoría académica, la aplicación reorientó su modelo de negocio. Se abandonó el intento de controlar alimentos perecederos (verduras, carnes), los cuales generaban una carga operativa insostenible, enfocando el desarrollo exclusivamente en artículos no perecederos. Esto transformó la herramienta en un exitoso auditor de alacena pre-compra.
- **Refactorización de Rutas:** Se aplicaron mejoras en el árbol de rutas (PublicRoute, ProtectedRoute) aislando la pantalla de recuperación de contraseña para evitar bloqueos por sesiones temporales de Supabase.

5.2. Mejoras propuestas para futuras versiones

La arquitectura prevee las siguientes escalabilidades técnicas:

- **Integración de Escáner de Código de Barras:** Implementación de acceso a la cámara para leer códigos EAN/UPC y conectarlo a la API Open Food Facts.
- **Aplicación Móvil Nativa (React Native):** Migración del frontend para permitir notificaciones *Push* reales de alertas de caducidad en iOS/Android.
- **Sincronización de Precios (Web Scraping):** Integración con APIs de supermercados para calcular el costo exacto de la lista de la compra en tiempo real.
- **Inteligencia Artificial Predictiva:** Modelos de Machine Learning basados en el histórico de la familia para sugerir listas predictivas.

6. Conclusiones del Proyecto

SmartPantry ha logrado con éxito todas las metas que se había planteado para su desarrollo. La separación clara de responsabilidades mediante una arquitectura Serverless (React y Supabase) y la implementación rigurosa del Patrón Repositorio han dado como resultado un sistema altamente escalable, seguro y fácil de mantener.

La opción estratégica de concentrarse únicamente en productos no perecederos se ha revelado como el principal acierto del proyecto. Al minimizar el esfuerzo de registro diario, SmartPantry evolucionó de un simple inventario a un verdadero auditor de inteligencia financiera familiar, demostrando ser una solución viable, tanto técnica como comercialmente, para el control económico del hogar.

7. Bibliografía

- Documentación oficial de React: <https://react.dev/>
- Documentación oficial de Supabase: <https://supabase.com/docs>
- Vite Next Generation Frontend Tooling: <https://vitejs.dev/guide/>
- Tailwind CSS Framework: <https://tailwindcss.com/docs>
- Zustand State Management: <https://zustand-demo.pmnd.rs/>
- Patrones de Diseño de Software (Repositorio): Freeman, E., & Robson, E. (2004). *Head First Design Patterns*. O'Reilly Media.